The complete AI engineer interview prep guide: 20 topics with resources

This guide covers every topic from a proptech AI Engineer job description — with handpicked 2024–2025 learning resources (crash courses, structured courses, and articles) plus interview-ready technical explanations for each. The resources prioritize practical, hands-on content over pure theory, and the technical explanations are calibrated for real interview depth.

The guide is organized into five thematic clusters: LLM & RAG systems, traditional ML & deep learning, MLOps & engineering, cloud & tools, and data foundations. Each topic includes the fastest path to competency (YouTube crash courses under 2 hours), the best deep-dive resources, and a concise but rigorous technical explanation you can adapt for interviews.

---

1. RAG pipelines: the most in-demand skill in applied AI

Best crash courses: The gold standard is **"RAG From Scratch"** by Lance Martin on freeCodeCamp — a 14-part playlist (also available as a single ~2.5-hour video) covering indexing, retrieval, generation, query translation, RAPTOR, ColBERT, and Self-RAG. freeCodeCamp For a faster start, **James Briggs'** "Chatbots with RAG" (~45 min) walks through OpenAI + Pinecone + LangChain. His separate "Semantic Chunking for RAG" video (~30 min) is essential for understanding modern chunking strategies.

Best articles: The Synthimind blog post "RAG Optimization Strategies 2025" covers GraphRAG, Agentic RAG, and hybrid search. The Activeloop free course (learn.activeloop.ai/courses/rag) offers 25+ lessons with 10 practical projects. Activeloop

Structured course: DeepLearning.AI's **"Building and Evaluating Advanced RAG Applications"** (free, taught by Jerry Liu of LlamaIndex) and their **"Building Agentic RAG with LlamaIndex"** on Coursera cover production-grade RAG.

Interview explanation: RAG is an architectural pattern that grounds LLM responses in external knowledge Data Nucleus LlamaIndex through three stages: **Retrieve** (embed the user query and perform similarity search against a vector store of pre-indexed document chunks), **Augment** (inject retrieved passages into the LLM prompt as context), and **Generate** (the LLM produces a response grounded in retrieved context rather than relying solely on parametric knowledge). Obot AI The ingestion pipeline involves chunking strategies — **fixed-size** (split every N tokens with overlap), **recursive** (try paragraph → sentence → word boundaries), and **semantic chunking** (use embedding similarity to detect topic shifts). Each chunk is embedded and stored in a vector database. LlamaIndex At query time, approximate nearest-neighbor search retrieves top-k chunks, Rohan-paul and optionally a **reranker** (cross-encoder like Cohere Rerank) re-scores results before passing to the LLM. The 2025 state of the art includes **GraphRAG** (knowledge graphs for multi-hop reasoning), **Agentic RAG** (autonomous agent loops with self-correction), **hybrid search** (combining dense vectors with sparse BM25 via Reciprocal Rank Fusion), Synthimind and **HyDE** (generating hypothetical answers for doc-to-doc similarity matching). X

2. Prompt engineering techniques every AI engineer must know

Best crash courses: DeepLearning.AI's "ChatGPT Prompt Engineering for Developers" (free, 1.5 hours, co-taught by Andrew Ng and Isa Fulford from OpenAI) is the single best starting point. DeepLearning.AI For advanced techniques, **PromptHub's** Chain of Thought guide covers zero-shot CoT, few-shot CoT, Auto-CoT, and self-consistency prompting.

Best articles: The **DAIR.AI Prompt Engineering Guide** (promptingguide.ai) is the definitive open-source reference with 3M+ learners. GitHub **OpenAI's official prompt engineering guide** (platform.openai.com/docs/guides/prompt-engineering) provides production best practices. The **Learn Prompting** site (learnprompting.org) is rated 4.8 stars with 82K+ enrolled learners. Learn Prompting

Structured course: The **Vanderbilt University "Prompt Engineering Specialization"** on Coursera (3 courses, 18+ hours, rated 4.8) goes from fundamentals through advanced prompt patterns. Unite.AI

Interview explanation: Prompt engineering techniques form a hierarchy of increasing sophistication. **Zero-shot prompting** gives the model a task with no examples. **Few-shot prompting** includes input-output demonstrations. Microsoft Learn **Chain-of-thought (CoT)** prompting encourages the model to articulate intermediate reasoning steps, dramatically improving performance on math and multi-step reasoning Learnprompting — zero-shot CoT can be triggered simply by appending "Let's think step by step." Prompt Hub Beyond these fundamentals, production prompt engineering uses **system prompts** (persona, constraints, behavioral rules), **structured output** (JSON mode or schema-constrained generation), **ReAct prompting** (interleaving reasoning and tool-use actions), and **prompt chaining** (decomposing complex tasks into sequential sub-prompts). For reasoning models like o1/o3, high-level goals work better than granular step-by-step instructions, while GPT-class models benefit from explicit, detailed prompts. OpenAI

---

3. LLM APIs: OpenAI and Anthropic integration patterns

Best crash courses: freeCodeCamp/Dataquest's "Build an AI Chatbot with Python and the OpenAI API" covers conversation memory, token budgeting, and custom personalities. DeepLearning.AI's prompt engineering course includes hands-on API usage with Jupyter notebooks. DeepLearning.AI

Best articles: The **OpenAI Developer Quickstart** (platform.openai.com/docs/quickstart) and **Anthropic SDK Python** (github.com/anthropics/anthropic-sdk-python) are the authoritative references. Tilburg.ai's OpenAI API tutorial provides a beginner-friendly walkthrough covering GPT-4o, Whisper, and environment variable management. tilburg.ai

Interview explanation: Both APIs use a **messages-based chat format** with role-based messages (system, user, assistant). OpenAI uses client.chat.completions.create(), Dataquest while Anthropic uses client.messages.create() Bronsondunbar with a separate system parameter. **Function calling/tool use** is a critical capability: you define tool schemas as JSON, and the model returns structured tool call requests rather than text when it decides a tool

is needed. Both support **streaming** (Server-Sent Events for token-by-token output), GitHub **structured output** (JSON mode for guaranteed parseable responses), and require **token management** (understanding input/output counts, max_tokens limits, and cost implications). Production considerations include exponential backoff retry logic, async/await patterns for concurrent requests, Collabnix and batch APIs for high-volume workloads. Key difference: Claude offers **200K token context windows** Obot AI and emphasizes Constitutional AI safety; OpenAI offers reasoning models (o1, o3) and broader multimodal capabilities.

---

4. LLM agent frameworks: LangChain, LlamaIndex, and CrewAI compared

Best crash courses: James Briggs' "LangGraph 101" (~32 min) covers graph-based agent architecture. Class Central **DeepLearning.AI's "Multi AI Agent Systems with CrewAI"** (free short course by Andrew Ng) teaches multi-agent collaboration. **"Building Agentic RAG with LlamaIndex"** (free on Coursera, taught by LlamaIndex CEO Jerry Liu) covers router agents and tool calling. Coursera

Best articles: Turing.com's "A Detailed Comparison of Top 6 AI Agent Frameworks in 2025" and IBM Developer's comparison of CrewAI, LangGraph, and BeeAI provide balanced framework evaluations. IBM

Structured course: IBM's "Agentic AI with LangGraph, CrewAI, AutoGen and BeeAI" on Coursera covers all major frameworks with hands-on labs including design patterns for sequential, routing, and parallel agent architectures. Coursera

Interview explanation: Agent frameworks enable autonomous systems that can **plan** (decompose goals into sub-tasks), **use tools** (call APIs, search, execute code), and **maintain memory**. The core loop follows the **ReAct pattern**: the LLM reasons about what action to take, executes a tool, observes the result, and iterates until complete. **LangChain/LangGraph** uses a graph-based state machine for complex workflows Langwatch with conditional routing and human-in-the-loop support Codecademy +2 — best for production-grade systems requiring fine-grained control. **LlamaIndex** is data-first, optimized for RAG with superior data connectors and indexing Codecademy — best when the agent primarily searches and synthesizes from private data. Latenode **CrewAI** uses role-based multi-agent coordination where agents have defined roles and tools DataCamp Softcery — best for collaborative tasks requiring team-like delegation. Selection heuristic: LangGraph for branching business logic with guardrails, CrewAI for rapid multi-agent prototyping, LlamaIndex for data-heavy retrieval agents. Codecademy Medium

---

5. MLOps: from experiment to production

Best crash courses: DataTalks.Club's MLOps Zoomcamp (free, 6 modules + project, updated for 2025) covers experiment tracking with MLflow, orchestration with Mage, deployment, and monitoring. Datatalks **freeCodeCamp's "Learn MLOps by Creating a YouTube Sentiment Analyzer"** (~3 hours) builds a complete pipeline with MLflow, DVC, Docker, and AWS. freeCodeCamp

Best articles: Neptune.ai's "How to Learn MLOps in 2024" provides a curated roadmap. Neptune.ai neptune ML Journey's "Model Versioning Strategies: DVC vs MLflow vs Weights & Biases" offers deep tool comparisons.

Structured course: The **Coursera MLOps Specialization by DeepLearning.AI** (4 courses by Andrew Ng) is the gold standard. Neptune.ai **Made With ML** (madewithml.com) by Goku Mohandas is an excellent free alternative covering end-to-end MLOps with Ray/Anyscale. GitHub

Interview explanation: MLOps combines ML, DevOps, and data engineering to reliably deploy and maintain ML systems in production. The lifecycle consists of **data management** (versioning with DVC, quality checks), **model development** (experiment tracking with MLflow or Weights & Biases), **CI/CD** (automated testing, training, and deployment via GitHub Actions), and **monitoring** (detecting data drift, concept drift, and performance degradation using tools like Evidently AI). **MLflow** handles experiment tracking, model registry, and deployment. DagsHub **DVC** extends Git to version large datasets using content-addressable storage. Ambiata A mature setup implements **Continuous Training (CT)** — automatically retraining models when data drift is detected or performance drops below thresholds. A typical workflow: data scientists track experiments in MLflow, version datasets with DVC, push code triggering GitHub Actions for automated retraining, evaluation against baseline, containerization with Docker, and deployment with canary/blue-green strategies.

---

6. End-to-end ML pipeline design

Best crash courses: The **freeCodeCamp MLOps course** (~3 hours) and **DataTalks.Club MLOps Zoomcamp orchestration module** cover pipeline design end-to-end. **Krish Naik's** "End to End ML Project Implementation with Docker, GitHub Actions, and Deployment" GitHub walks through the complete lifecycle.

Best articles: Neptune.ai's "How to Build an End-To-End ML Pipeline" and ZenML's "Orchestration Showdown: Dagster vs Prefect vs Airflow" are essential reads for understanding pipeline tools.

Structured course: Courses 2 and 3 of the **Coursera MLOps Specialization** ("ML Data Lifecycle in Production" and "ML Modeling Pipelines in Production") cover this thoroughly.

Interview explanation: An ML pipeline automates the sequential stages of data ingestion, validation, feature engineering, training, evaluation, and deployment. Label Rer Production pipelines are divided into three sub-pipelines: the **data pipeline** (ingesting, cleaning, validating data), the **training pipeline** (feature engineering, model training, hyperparameter tuning, evaluation), and the **deployment pipeline** (model serving, A/B testing, monitoring). Neptune.ai **Orchestration tools** like Apache Airflow (DAG-based scheduling), Prefect (Python-native with @task/@flow decorators), and Kubeflow Pipelines (Kubernetes-native) manage dependencies, scheduling, and retries. Madewithml Best practices include data validation gates (Great Expectations), feature stores (Feast) for consistent feature computation across training and serving, and model registry integration (MLflow) for lifecycle management. Deployment strategies include **batch inference** (periodic prediction via scheduled pipelines) and **real-time inference** (REST API endpoints containerized with Docker, scaled with Kubernetes). DevOps.dev

7. Python ML frameworks: PyTorch, TensorFlow, and scikit-learn

Best crash courses: Daniel Bourke's "PyTorch for Deep Learning" on freeCodeCamp (~25.5 hours, companion book at learnpytorch.io) is the most comprehensive PyTorch resource. KDnuggets **Andrej Karpathy's "Neural Networks: Zero to Hero"** (~13 hours, 7 videos) is exceptional for understanding PyTorch internals and autograd from scratch. Andrej Karpathy For TensorFlow, freeCodeCamp's **"TensorFlow 2.0 Crash Course"** (~7 hours) covers CNNs, RNNs, NLP, and reinforcement learning.

Best articles: Sebastian Raschka's "PyTorch in One Hour" (sebastianraschka.com) is an excellent concise primer. Sebastian Raschka The official PyTorch tutorials (docs.pytorch.org/tutorials/) and PyTorch scikit-learn's Pipeline documentation are authoritative references.

Structured course: Daniel Bourke's Zero to Mastery PyTorch course (learnpytorch.io for the free book) for deep learning. Daniel Bourke Kaggle's free Intermediate ML course for scikit-learn.

Interview explanation: PyTorch uses a dynamic "define-by-run" computation graph built during the forward pass, enabling intuitive debugging with standard Python tools. Sebastian Raschka Core abstractions include torch.Tensor (GPU-accelerated with autograd), nn.Module for model definition, Medium DataLoader for batched data handling, and torch.compile() (PyTorch 2.0+) for graph-mode optimization. PyTorch dominates research (~60% of papers). DataCamp Daniel Bourke **TensorFlow/Keras** provides three API levels: Sequential (simple stacking), Functional (DAG-based), and Subclassing (full flexibility). Its production ecosystem (TF Serving, TF Lite, TF.js) remains a strength. **scikit-learn's** Pipeline chains transformers and estimators into a single object — crucial for preventing data leakage in cross-validation. scikit-learn Custom transformers inherit from BaseEstimator + TransformerMixin, Andrewvillazon while ColumnTransformer handles heterogeneous data types. GridSearchCV tunes hyperparameters across entire pipelines using the step__param naming convention. scikit-learn

---

8. Vector databases: FAISS, Pinecone, Chroma, and Weaviate

Best crash courses: James Briggs' Pinecone/Vector Search series on YouTube covers practical implementations with Pinecone, Cohere, and CLIP. **Pinecone's Learning Center** (pinecone.io/learn) has excellent written and video content explaining FAISS, HNSW, IVF, and similarity search fundamentals.

Best articles: Firecrawl's "Best Vector Databases in 2025: A Complete Comparison Guide" evaluates 14 databases with real VectorDBBench numbers. Pinecone's "Introduction to Facebook AI Similarity Search (Faiss)" is the best FAISS technical deep-dive, covering IndexFlatL2, IVF, and Product Quantization.

Structured course: DeepLearning.AI's **"Building Applications with Vector Databases"** and **"Advanced Retrieval for AI with Chroma"** (both free short courses).

Interview explanation: Vector databases store high-dimensional embedding vectors and enable fast similarity search using **ANN (Approximate Nearest Neighbor)** algorithms. The two main algorithm families are **HNSW** (graph-based, O(log n) queries, excellent recall, higher memory) (Firecrawl) and **IVF** (partition-based using k-means clustering, faster for very large datasets). (Pinecone) Similarity metrics include **cosine similarity** (most common for normalized embeddings), **L2/Euclidean distance**, and **dot product**. **Product Quantization (PQ)** compresses vectors by encoding sub-vectors with centroid IDs, reducing memory at some accuracy cost. (Pinecone)

The key trade-offs between databases:

- **FAISS** (Meta): A library, not a database — no persistence or API. Maximum flexibility, GPU acceleration. Best for research or custom billion-scale pipelines (LiquidMetal AI)
- **Pinecone**: Fully managed, serverless, sub-50ms latency at billion scale, (sparkco ai) SOC 2 certified. Best for zero-ops production. Premium pricing (Aloa)
- **Chroma**: Open-source, minimal setup. Best for prototyping and small/medium applications (Shaped)
- **Weaviate**: Open-source, native hybrid search (dense + BM25), GraphQL API, modular vectorizers. Best for complex data relationships + vector search (Shaped) (MLOps World)
- **Qdrant**: Rust-based, excellent metadata filtering, cost-efficient. Best for production workloads needing filtering (Shaped)

Quick decision: Prototyping → Chroma. Zero-ops production → Pinecone. Hybrid search → Weaviate or Qdrant. (Aloa) Billion-scale → Milvus or FAISS.

---

9. Cloud ML platforms: SageMaker, Vertex AI, and Azure ML

Best crash courses: Krish Naik's AWS SageMaker playlist (7-part series, (Class Central) 15–30 min each) covers build-train-deploy on SageMaker with XGBoost. His **"Complete MLOps Bootcamp"** on Udemy (50+ hours) includes SageMaker deployment, MLflow, DVC, and Docker. (Udemy)

Best articles: Ankur's Newsletter compares all three platforms with 2025 pricing data and hardware specs. CloudExpat's "In-Depth Comparison: AWS SageMaker, Azure ML, and GCP Vertex AI in 2024" provides enterprise-focused analysis.

Structured course: For certification, the **AWS Skill Builder Exam Prep Plan** (free) covers the ML Specialty exam. (AWS) **Frank Kane/Stephane Maarek's Udemy course** "AWS Certified Machine Learning Specialty" is widely recommended by exam passers. (Medium) Note: AWS ML Specialty (MLS-C01) retires March 31, 2026 (AWS) — the newer **ML Engineer Associate** is also available.

Interview explanation: Cloud ML platforms provide managed environments for the full ML lifecycle. **AWS SageMaker** (34% market share) leverages Inferentia chips for cost-efficient inference and HyperPod for distributed training at massive scale (100B+ parameter models). **GCP Vertex AI** (22%) offers TPU v5p clusters

and deep BigQuery integration for petabyte-scale analytics. **Azure ML** (29%) excels in regulated industries with confidential computing (Ankur's Newsletter) and Microsoft 365/Purview governance integration. Key concepts across platforms include managed training jobs (ephemeral compute that auto-provisions/releases), real-time vs batch inference endpoints, Feature Stores for consistent feature serving, AutoML, and MLOps pipeline orchestration. Platform selection typically follows existing cloud infrastructure. (TechTarget)

---

10. NLP and Arabic language processing

Best crash courses: The **Hugging Face NLP Course** (YouTube playlist, chapters 1–9) covers Transformers, tokenizers, datasets, and fine-tuning (Hugging Face) — continuously updated. **Andrej Karpathy's "Let's Build GPT"** (~2 hours) builds a GPT from scratch and is the best resource for deeply understanding the Transformer architecture.

Best articles: The **Hugging Face LLM/NLP Course** (huggingface.co/learn/llm-course/) is the definitive free resource. For Arabic NLP: **CAMeL Tools** (github.com/CAMeL-Lab/camel_tools) from NYU Abu Dhabi (GitHub) handles morphological modeling, dialect identification, NER, and sentiment analysis. (ACL Anthology) **AraBERT** (github.com/aub-mind/arabert) achieves SOTA on Arabic benchmarks. (GitHub) **CAMeLBERT** provides 8 pre-trained BERT models for MSA, dialectal, and classical Arabic. (GitHub)

Structured course: The **Hugging Face NLP Course** (free) for Transformers. (Hugging Face) DeepLearning.AI's **"NLP Specialization"** on Coursera for traditional NLP foundations. For Arabic NLP, start with CAMeL Tools documentation + AraBERT model cards on Hugging Face, then practice fine-tuning on Arabic sentiment datasets (ArSAS, ASTD). (GitHub)

Interview explanation: Modern NLP revolves around the **Transformer architecture** — self-attention computes pairwise relationships between all tokens simultaneously, enabling massive parallelization. (Udacity) (Pinecone) The dominant paradigm is **pre-training + fine-tuning**: models are pre-trained with self-supervised objectives (masked language modeling for BERT-style encoders, next-token prediction for GPT-style decoders), then fine-tuned on task-specific data. (Hugging Face) The Hugging Face `transformers` library provides `pipeline()` for zero-code inference, (Hugging Face) `AutoModel`/`AutoTokenizer` for model loading, and the `Trainer` API for fine-tuning.

Arabic NLP presents unique challenges: morphological richness (highly inflected root-pattern system), orthographic ambiguity (short vowels typically omitted), and dialectal variation (MSA differs significantly from Egyptian, Gulf, and Levantine dialects). (ACL Anthology) **AraBERT** is pre-trained on ~77GB of Arabic text. **CAMeL Tools** provides normalization, morphological analysis, dialect identification, NER, and sentiment analysis. Best practice: preprocess with CAMeL Tools (normalize, remove diacritics), then fine-tune AraBERT or CAMeLBERT on task-specific data. Language-specific models consistently outperform multilingual models (mBERT, XLM-RoBERTa) on Arabic tasks. (GitHub)

---

11. Computer vision: CNNs, transfer learning, and Vision Transformers

Best crash courses: Daniel Bourke's PyTorch course (sections 03 and 06 on Computer Vision and Transfer Learning) builds a FoodVision project with CNNs and transfer learning hands-on. Zero To Mastery Zero to Mastery **Stanford CS231n** (cs231n.stanford.edu) remains the gold standard for theoretical foundations CS231n — lecture videos from past years are freely available.

Best articles: Analytics Vidhya's "Top 30+ Computer Vision Models For 2026" provides a comprehensive timeline from LeNet through ViT and SAM. Viso.ai's "Vision Transformer: A New Era in Image Recognition" clearly explains ViT architecture.

Structured course: Stanford CS231n for theory. The **Hugging Face Computer Vision Course** (free, huggingface.co/learn/computer-vision-course/) for modern ViT fine-tuning approaches.

Interview explanation: CNNs use convolutional layers (learnable filters detecting spatial features), pooling layers (downsampling for translation invariance), and fully connected layers for classification. GeeksforGeeks Key architectures: ResNet (skip connections enabling 100+ layer training), EfficientNet (compound scaling via NAS). **Transfer learning** loads a pre-trained backbone (trained on ImageNet's 1.2M images), replaces the classification head, and fine-tunes Hugging Face — either freezing early layers (feature extraction for small datasets) or training all layers with discriminative learning rates (lower LR for earlier layers). Label Your Data

Vision Transformers (ViT) split images into fixed-size patches (e.g., 16×16), linearly project each to an embedding, add positional embeddings, prepend a [CLS] token, and process through standard Transformer encoder blocks with multi-head self-attention. UvA DL Notebooks ViTs lack CNN inductive biases (locality, translation equivariance) Label Your Data and require large-scale pre-training to outperform CNNs, but scale better once pre-trained. Hugging Face Modern approaches include **CLIP** (contrastive language-image pre-training for zero-shot classification), Hugging Face **DINOv2** (self-supervised ViT), and **Swin Transformer** (hierarchical ViT with shifted windows).

---

12. LangChain: chains, agents, memory, and LangGraph

Best crash courses: James Briggs' "LangChain Mastery in 2025" (5 hours, 10 chapters) is the single best LangChain YouTube resource — covers LCEL, LangSmith, prompt templates, memory types, agents, streaming, and async patterns. Class Central **DeepLearning.AI's "LangChain for LLM Application Development"** (free, 1 hour, by Harrison Chase and Andrew Ng) is the fastest onramp. DeepLearning.AI **Sam Witteveen's** LangChain playlist (50+ videos) provides granular topic-by-topic coverage. GitHub

Best articles: LangChain's official documentation (python.langchain.com) covers LCEL, agents, and LangGraph integration. DataCamp's "LangChain vs LangGraph vs LangSmith vs LangFlow" maps the entire ecosystem.

Structured course: DeepLearning.AI's short course (free). **LangChain Academy's "Introduction to LangGraph"** (free, official) for agent orchestration. LangChain

Interview explanation: LangChain's core innovation is **LCEL (LangChain Expression Language)** — a pipe-based composition syntax (`prompt | model | parser`) that makes chains declarative and supports streaming/async natively. DataCamp Key components: Prompt Templates (dynamic prompt construction), Chat Models (unified interface to swap providers), Memory (Buffer/Window/Summary for maintaining context), Retrievers (vector store integration for RAG), Tools, and Agents. Medium The ecosystem has expanded: **LangGraph** adds graph-based stateful orchestration with nodes, edges, conditional routing, checkpointing, and human-in-the-loop Medium AlphaCorp AI — essential for production agents. Medium LangChain **LangSmith** provides tracing, evaluation, and monitoring. In 2025, LangChain introduced **Deep Agents** — batteries-included agents with conversation compression and sub-agent spawning, built on LangGraph. LangChain Recommended progression: LangChain for linear workflows → LangGraph for complex stateful agents → LangSmith for production observability. DataCamp

---

13. LlamaIndex: the data-first RAG framework

Best crash courses: DeepLearning.AI's "Building Agentic RAG with LlamaIndex" (free on Coursera, taught by Jerry Liu) covers router agents, tool calling, and multi-document agents. Coursera **echohive's** "LlamaIndex Step by Step Introduction" on YouTube is a solid beginner-friendly guide.

Best articles: The **LlamaIndex Advanced RAG Cheat Sheet** (llamaindex.ai/blog) covers chunk-size optimization, iterative retrieval, and advanced techniques. **Real Python's "LlamaIndex in Python"** provides excellent hands-on examples. The Hugging Face Agents Course includes a dedicated LlamaIndex module. Hugging Face

Structured course: DeepLearning.AI's short course (free). **Educative's "Mastering LlamaIndex"** covers RAG, structured extraction, agents/workflows, and monitoring comprehensively. Educative

Interview explanation: LlamaIndex follows a clear pipeline: **Data Connectors** (ingest from PDFs, APIs, databases via LlamaHub), **Documents** parsed into **Nodes** (the atomic retrieval unit with source-linking metadata), **Indexes** (primarily `VectorStoreIndex` generating and storing embeddings), **Retrievers** (similarity search), **Query Engines** (combining retrieval with LLM generation), and **Response Synthesizers** (merging context with queries). Medium Llamaindex What distinguishes LlamaIndex is its data-first design: built-in tools for semantic chunking, hybrid retrieval (BM25 + vector), reranking with cross-encoders, and query decomposition. Medium **LlamaParse** handles complex PDFs with tables and charts. For agents, `QueryEngineTool` exposes retrieval as a tool to ReAct-style agents that plan and synthesize across multiple document collections. Medium Many production teams combine LlamaIndex for indexing/retrieval with LangGraph for broader orchestration. Galileo AI

---

14. DevOps for ML engineers: Docker, Git, CI/CD, and Kubernetes

Best crash courses: TechWorld with Nana's "Docker Tutorial for Beginners" (~3 hours) is widely regarded as

the best Docker tutorial on YouTube. Her **"Kubernetes Crash Course for Absolute Beginners"** covers Pods, Services, Deployments, and Minikube. **Krish Naik's** "Complete Docker for Data Science" and "Deploy End to End ML Projects with CI/CD Pipeline" (Class Central) are specifically tailored for ML engineers.

Best articles: DataCamp's "A Beginner's Guide to CI/CD for Machine Learning" provides step-by-step GitHub Actions + CML integration. (DataCamp) **Made With ML's** CI/CD section (madewithml.com) covers advanced GitHub Actions workflows for ML with automated evaluation reporting on PRs. (GitHub) (Madewithml)

Structured course: Course 4 of the **Coursera MLOps Specialization** ("Deploying ML Models in Production") covers containerization, Kubernetes scaling, and monitoring. (Forecastegy) **DataCamp's "CI/CD for Machine Learning"** is a focused interactive course.

Interview explanation: Docker packages a model with its exact environment into portable container images, eliminating environment inconsistencies. (DevOps.dev) (KDnuggets) A typical ML Dockerfile starts from a Python base image, installs dependencies, copies model artifacts and serving code, and runs a serving framework (FastAPI, TorchServe). **Git** manages code versioning while **DVC** extends Git to handle large datasets and model files. **CI/CD for ML** adds Continuous Training: a GitHub Actions workflow triggers on push, runs data validation, trains the model, evaluates against baseline, builds a Docker image, pushes to a container registry (ECR, Artifact Registry), and deploys. (DataCamp) (Madewithml) **CML** (Continuous Machine Learning) posts training metrics directly on pull requests. (Medium) **Kubernetes** manages multiple container replicas with load balancing, automatic restarts, and auto-scaling. (DevOps.dev) **KServe** provides Kubernetes-native ML model serving with canary deployments, A/B testing, and GPU auto-scaling.

---

15. Advanced data analysis: pandas, NumPy, and feature engineering

Best crash courses: Krish Naik's comprehensive playlists on pandas, NumPy, and feature engineering (part of his Complete Data Science series) are thorough. For NumPy specifically, the "NumPy Data Science Essential Training" (~12 hours) covers broadcasting, structured arrays, and EDA. (Class Central)

Best articles: Wes McKinney's "Python for Data Analysis, 3rd Edition" (wesmckinney.com/book/, free online) is the authoritative reference by the creator of pandas. Real Python's "Pythonic Data Cleaning With pandas and NumPy" covers practical cleaning patterns. GeeksforGeeks' "Advanced Feature Engineering with Pandas" covers datetime features, binning, interaction terms, and encoding. (GeeksforGeeks)

Structured course: Udemy's "The Complete Pandas Bootcamp 2025" by Alexander Hagmann (36+ hours, 150+ exercises, covers Pandas 2.2). (Udemy) **Kaggle Learn's** free "Feature Engineering" and "Data Cleaning" micro-courses for focused practice.

Interview explanation: NumPy's ndarray is a contiguous block of typed memory enabling vectorized operations orders of magnitude faster than Python loops. Key advanced concepts include broadcasting (automatic shape alignment), structured arrays, and memory-efficient views vs copies. **Pandas** provides the DataFrame abstraction with critical ML capabilities: method chaining with (.pipe()) for composable

preprocessing pipelines, Taylor-amarel pd.Categorical for memory-efficient encoding, multi-index for hierarchical data, and eval()/query() for performant large-DataFrame operations.

Feature engineering strategies include: temporal features (year/month/day-of- CodeSignal week/is_weekend), rolling/expanding window aggregations for time series, interaction features (products/ratios), target encoding (replacing categorical values with mean target — with cross-validation to prevent leakage), frequency encoding, and polynomial features. Data cleaning best practices: understand MCAR/MAR/MNAR missing data patterns before choosing imputation, detect outliers via Z-scores or IQR, and encapsulate all steps in sklearn Pipelines to prevent train-test leakage.

---

16. Text embeddings: from theory to multilingual production

Best crash courses: James Briggs' Sentence Transformers series on YouTube is canonical for understanding embeddings. His "Supercharge eCommerce Search" video (~35 min) covers multimodal embeddings with CLIP and hybrid sparse/dense search.

Best articles: OpenAI's January 2024 blog post "New Embedding Models and API Updates" announced text-embedding-3-small and text-embedding-3-large with the new dimensions parameter. **Pinecone's "Sentence Transformers: Meanings in Disguise"** (pinecone.io/learn) explains the evolution from word2vec → BERT → SBERT. For Arabic: the **Omartificial-Intelligence-Space Arabic Matryoshka Models** on Hugging Face achieve #1 on STS17 Arabic MTEB leaderboard (score: 85.3).

Structured course: DeepLearning.AI's **"Vector Databases: from Embeddings to Applications"** (free) and DataCamp's **"Introduction to Embeddings with the OpenAI API."**

Interview explanation: Text embeddings are dense vector representations capturing semantic meaning — similar texts have high cosine similarity. **Sentence-Transformers (SBERT)** fine-tunes BERT/MPNet using Siamese networks with contrastive loss, producing fixed-size sentence embeddings via mean pooling. OpenAI's latest models are **text-embedding-3-small** (1536 dims, $0.02/1M tokens) and **text-embedding-3-large** (3072 dims, $0.13/1M tokens). A key innovation is the dimensions parameter: you can truncate embeddings (e.g., 3072→1024) without significant quality loss thanks to **Matryoshka Representation Learning** — the model is trained so leading dimensions carry the most important information. A text-embedding-3-large at 256 dimensions still outperforms the older ada-002 at 1536 dimensions.

For **multilingual/Arabic** embeddings, key models include **multilingual-e5-large** (strong cross-lingual performance), **paraphrase-multilingual-mpnet-base-v2** (50+ languages), and Google's **LaBSE** (109 languages). State-of-the-art Arabic-specific models include **Swan** (dialect-aware), **GATE** (Matryoshka + hybrid loss), and the Arabic Matryoshka models on Hugging Face. Fine-tuning multilingual models on domain-specific Arabic data can yield **5–10% retrieval improvements**.

---

17. FastAPI: the standard for ML model serving

Best crash courses: freeCodeCamp's "Python FastAPI Crash Course" by Tomi Tokko (~1 hour) covers fundamentals and deployment. For ML-specific serving, the JetBrains PyCharm blog tutorial "How to Use FastAPI for Machine Learning" walks through building an ML API prototype with KNeighborsClassifier.

Best articles: MachineLearningMastery's "Step-by-Step Guide to Deploying ML Models with FastAPI and Docker" builds a Random Forest predictor with production patterns. **EvidentlyAI's** "ML serving and monitoring with FastAPI and Evidently" shows a full architecture: FastAPI + PostgreSQL + Evidently monitoring + Docker Compose. The **FastAPI official tutorial** (fastapi.tiangolo.com) is considered one of the best framework documentations available and doubles as a free course.

Structured course: Udemy's "Deploy ML Model in Production with FastAPI and Docker" covers AWS deployment (EC2, S3, ECR, Fargate), load testing, and real-world projects including sentiment analysis and pose estimation.

Interview explanation: FastAPI has become the de facto standard for serving ML models, with adoption at **31% among data scientists**. Built on Starlette (ASGI) and Pydantic, it achieves performance comparable to Node.js and Go. Key features for ML serving: **Pydantic validation** (define request/response schemas as typed Python classes, automatically rejecting malformed requests), **async support** (`async def` endpoints for concurrent inference requests), **automatic OpenAPI docs** (Swagger UI at `/docs`), and **dependency injection** (clean model loading and authentication). A typical pattern: load the model at startup, define a Pydantic `BaseModel` for input features, create a `POST /predict` endpoint running `model.predict()`, and deploy with Uvicorn behind Gunicorn with multiple workers, containerized in Docker.

---

18. Power BI for data visualization

Best crash courses: Guy in a Cube (Adam Saxton & Patrick LeBlanc, 525K+ subscribers, both Microsoft employees) is the premier Power BI channel. **SQLBI** (Marco Russo & Alberto Ferrari, 126K subscribers) is the gold standard for DAX and data modeling — their "DAX 101" series covers fundamentals rigorously. **Simon Sez IT's** "Microsoft Power BI for Beginners" (~2 hours) is a comprehensive single-video crash course.

Best articles: SQLBI.com has 200+ free articles on DAX and data modeling plus free video courses "Introducing DAX" and "Introduction to Data Modeling." Microsoft Learn (learn.microsoft.com/power-bi/) provides official learning paths.

Structured course: Prepare for the **Microsoft PL-300 certification** ("Power BI Data Analyst Associate") via free Microsoft Learn modules. **SQLBI's** "Mastering DAX" and "Optimizing DAX" for expert-level depth.

Interview explanation: Power BI's core technical concepts include **DAX (Data Analysis Expressions)** — a formula language operating on a columnar in-memory engine (VertiPaq) enabling sub-second queries on millions of rows. Key DAX concepts: evaluation context (row context vs filter context), iterators (SUMX, AVERAGEX), and time intelligence (TOTALYTD, SAMEPERIODLASTYEAR). **Data modeling** follows a

star schema with fact and dimension tables. **Power Query (M language)** handles ETL from 100+ data sources. For AI engineers, Power BI integrates with Python/R scripts for custom visualizations, connects to Azure ML for real-time scoring, and supports streaming datasets for live model monitoring dashboards. Understanding **DirectQuery** (live database connection) vs **Import mode** (in-memory engine), refresh strategies, and row-level security matters for enterprise deployments.

---

19. n8n workflow automation with AI integration

Best crash courses: Data Science with Harshit's "n8n Crash Course" (~3 hours) covers setup, lead management, web scraping, AI agents, and multi-agent systems. The **n8n Official YouTube Channel** has a 9-part beginner course from "Introduction to Automation" through advanced workflows. **AI Automation Society** on YouTube covers voice AI agents, HeyGen integration, and MCP server workflows.

Best articles: The **n8n official docs tutorial** "Build an AI workflow in n8n" (docs.n8n.io/advanced-ai/intro-tutorial/) is the authoritative starting point. Exascale AI's "How to Define an AI Workflow in n8n (2025)" covers AI Agent nodes, vector stores, and LangChain integration.

Structured course: n8n's **official video courses** (docs.n8n.io/video-courses/) are free. The **n8n AI Starter Kit** provides ready-to-use AI workflow templates, and the community library offers **600+ templates**.

Interview explanation: n8n is an open-source workflow automation platform with 500+ integrations, distinguishing itself from Zapier/Make through complex branching, looping, code injection (JavaScript/Python), and self-hosting for data privacy. For AI engineering, n8n provides dedicated AI nodes integrating with OpenAI, Gemini, and other LLMs, plus a LangChain-based agent framework. Key components: Trigger Nodes (webhooks, schedules), AI Agent Nodes (LLM orchestrators with tool use), Memory & Vector Store nodes (for RAG), and Action Nodes (email, database, API calls). n8n supports **MCP (Model Context Protocol)** server triggers for external AI system integration. At **$50/month** (10,000 executions, unlimited users) vs Zapier's $170/month, n8n offers significant cost advantages while the self-hosting option provides complete infrastructure control.

---

20. Gradient boosting: XGBoost and LightGBM for interviews

Best crash courses: StatQuest with Josh Starmer is the undisputed champion here — his **Gradient Boost series** (4 parts) and **XGBoost series** (4 parts) use exceptional visual explanations universally recommended for interview prep. His "XGBoost in Python, from Start-to-Finish" adds hands-on implementation. He also covers CatBoost's Ordered Target Encoding.

Best articles: The **XGBoost official documentation** "Introduction to Boosted Trees" provides the formal derivation. **Neptune.ai's** "XGBoost vs LightGBM: How Are They Different" covers depth-wise vs leaf-wise growth, GOSS, and EFB. Machine Learning Mastery's comparison provides practical Python code for all implementations.

Structured course: StatQuest's full series (free, ~3 hours total). **Kaggle's "Intermediate Machine Learning"** micro-course (free) for hands-on XGBoost practice.

Interview explanation: Gradient boosting sequentially builds weak learners (shallow trees) where each new tree corrects the ensemble's errors by fitting to **pseudo-residuals** (negative gradient of the loss function). The "gradient" refers to gradient descent in function space — adding functions that point in the steepest descent direction. This framework is loss-agnostic: any differentiable loss function works.

XGBoost improves upon vanilla gradient boosting with: **second-order approximation** (using both gradient and Hessian for Newton-Raphson optimization), **regularization** (L1/L2 on leaf weights plus tree complexity penalty in the objective), **sparsity-aware split finding** (automatic missing value handling), and **weighted quantile sketch** (efficient split-point finding). Trees grow **depth-wise** (level by level). Key hyperparameters: `learning_rate` (0.01–0.3), `max_depth` (3–10), `n_estimators`, `subsample`, `colsample_bytree`.

LightGBM (Microsoft) differs with: **leaf-wise tree growth** (grows the leaf with highest loss reduction — deeper asymmetric trees, lower loss with fewer splits, but risks overfitting on small data), **GOSS** (keeps large-gradient instances while sampling small-gradient ones), and **EFB** (bundles mutually exclusive features). LightGBM is typically **5–10x faster** than XGBoost. **CatBoost** (Yandex) specializes in Ordered Target Statistics for categorical features without manual encoding. Selection guide: XGBoost for maximum community support; LightGBM for large datasets where speed matters; CatBoost when data has many categorical features or minimal tuning is desired.

Conclusion: a prioritized learning roadmap

The 20 topics above span a wide range, but they cluster around a clear priority order for a proptech AI Engineer role. **RAG pipelines, LangChain/LlamaIndex, and prompt engineering** are the highest-signal skills — they dominate current AI engineering interviews and have the most immediately applicable value. Vector databases and embeddings are tightly coupled with RAG and should be learned together. The LLM ecosystem (APIs, agents) forms the second tier of priority.

Traditional ML skills (gradient boosting, scikit-learn pipelines, feature engineering) remain essential for interview algorithm rounds and for the many production tasks where LLMs are overkill. MLOps and DevOps (Docker, CI/CD, MLflow) are what separate a notebook experimenter from a production engineer — these are differentiators in interviews.

Three free resources stand above the rest in breadth and quality: **DeepLearning.AI's short courses** (free, covering RAG, LangChain, agents, and prompt engineering), **DataTalks.Club's MLOps Zoomcamp** (free, end-to-end MLOps), and **Hugging Face's NLP/LLM Course** (free, the definitive Transformers resource). Starting with these three platforms covers roughly 70% of the topics in this guide. StatQuest remains irreplaceable for building intuitive understanding of ML algorithms, and James Briggs' YouTube channel is the single best source for the LLM/RAG/vector search ecosystem.